DALÓ / CAMPUS EVENTS ANDROID MVP

Daló

Student event registration and QR attendance

Daló replaces paper sign-up sheets and door queues with one simple flow: students discover campus events, register in the app, and present a unique QR ticket for an event manager to confirm attendance.

Platform	Backend	Operating model
Flutter for Android	Firebase Authentication + Cloud Firestore	Firebase Spark / zero budget

What the app supports

01 Accounts	Verified student-email sign-up, sign-in, email verification, and password reset.
02 Events	Browse events, host an event, register while seats remain, and follow live attendee counts.
03 Tickets	Get an event-specific QR ticket in the app and save or share it.
04 Check-in	Scan inside the app with the camera or find a registered attendee by name and check them in manually.

Project goal

Make campus registration and attendance quick to understand for students and event hosts, while keeping the implementation small enough to explain and maintain as a college project.

How a student uses Daló

The flow begins with a verified institution account and ends with an event-specific ticket.

Step	Student action	What Daló does
1. Create account	Enter name, institution email, and password.	Creates a student account and sends Firebase's email-verification link.
2. Verify and sign in	Open the email link, then sign in.	Checks the institution domain and verified-email state, then loads the user's Daló profile.
3. Discover	Search events by title or location.	Shows upcoming events with date, place, availability, and registration count.
4. Register	Open an event and choose to register.	Checks for an existing ticket and remaining capacity, then creates the registration and increments the event count together.
5. Show ticket	Open Tickets or the event's QR ticket.	Displays the high-contrast QR token for that event registration; the ticket can be saved or shared.
6. Attend	Present the QR code at the entrance.	The event owner or an authorized facilitator scans it in Daló; a successful check-in is recorded once.

Who can do what

Role	Typical access
Student account	Browse and join any open event; view and share the account's own tickets.
Event owner	Any verified institution account may host. The creator manages only their event's attendee list and check-in.
Facilitator	Optional administrator-assigned role for check-in across events. Public sign-up cannot grant this role.

Design direction

Daló uses warm off-white and white surfaces, dark readable type, restrained coral accents, rounded event cards, and direct navigation. The visual approach is event-first and inspired by modern event apps, while the screens and identity remain Daló's own.

How registration and check-in stay consistent

The key design choice is to write the ticket and the event count as one Firestore transaction.

Registration lifecycle

- Daló creates a cryptographically random 32-byte token and encodes it as a URL-safe string. The QR contains only this opaque token, not a student's email or name.
- A registration is identified by the event and student account. A Firestore transaction reads the current registration and event, refuses duplicates or full events, then creates the registration and increments the count atomically.
- Firestore Security Rules use the before/after transaction state to require that both writes match. This helps prevent capacity overbooking on the Spark plan.
- At the door, the in-app camera reads the token for the selected event. Another Firestore transaction changes the registration from registered to checked in. Repeat scans receive an already-checked-in result.
- Manual check-in uses the same one-time status update after the host searches the attendee list.

Firestore records

Collection	Document content (main fields)
users/{uid}	email, displayName, role, createdAt
events/{eventId}	title, date, time, location, capacity, registeredCount, organizerUid, organizerName, createdAt
registrations/{eventId_uid}	eventId, studentUid, studentName, studentEmail, qrToken, status, registeredAt, checkedInAt

Access and safety

- The current accepted domains are @udd.edu.ph and @cdd.edu.ph; sign-up and Firestore rules should stay aligned.
- Firestore rules restrict event management to the event creator, with optional admin-assigned facilitator access. Users cannot assign themselves elevated roles.
- The Spark design runs transactions in the app and relies on Firestore Security Rules. It is appropriate for a college MVP, but is less resistant to modified clients than a trusted server backend.

Deployment reminder

The currently deployed Firestore rules must match the repository's latest firestore.rules. When rules change, deploy with firebase deploy --only firestore from the repository root. This deploys Firestore rules/indexes only; it does not deploy Cloud Functions.

Code map: Flutter application

The app lives in dalo/. Screens handle presentation; services handle Firebase operations; models translate Firestore documents into Dart objects.

Path	Purpose
dalo/lib/main.dart	Starts Flutter, sets portrait orientation, and initializes Firebase.
dalo/lib/app.dart	Creates the Material app and applies the shared Daló theme.
dalo/lib/core/theme/dalo_theme.dart	Color palette, typography, buttons, fields, cards, and navigation styling.
dalo/lib/core/auth/auth_gate.dart	Listens to sign-in state, waits for the user profile, routes to home, and reports profile loading errors.
dalo/lib/core/widgets/common_widgets.dart	Reusable cards, primary/secondary buttons, loading/empty states, error banners, and status badges.
dalo/lib/models/app_user.dart	Account profile fields and student/organizer/facilitator role model.
dalo/lib/models/event.dart	Event fields, capacity status, and Firestore conversion.
dalo/lib/models/registration.dart	Ticket fields, QR token, registration status, and check-in time.
dalo/lib/services/auth_service.dart	Student email-domain validation, Firebase sign-up/sign-in, verification, profile stream, and password reset.
dalo/lib/services/event_service.dart	Creates events and provides event list/detail streams from Firestore.
dalo/lib/services/registration_service.dart	Capacity-safe registration, random QR tokens, student tickets, QR lookup, and transactional check-in.
dalo/lib/screens/auth/	Login, account creation, verification notice, and password reset screens.
dalo/lib/screens/student/	Event discovery, event details/registration, My Tickets, and QR ticket display/share.
dalo/lib/screens/organizer/	Create Event, hosted-event tools, live attendee count, attendee list, scan/manual check-in entry points.
dalo/lib/screens/facilitator/	Facilitator event list, in-app camera scanner, and attendee-name search/manual check-in.
dalo/pubspec.yaml + pubspec.lock	Declare Flutter packages and lock resolved dependency versions.

Code map: Firebase, Android, and project notes

These files connect the Flutter client to Firebase, constrain database access, and explain how to build the Android package.

Path	Purpose
firestore.rules	Server-enforced access rules for verified accounts, event ownership, capacity-safe registration, and one-time check-in.
firestore.indexes.json	Composite indexes for ticket and event-attendee queries.
firebase.json	Firebase CLI mapping for Firestore rules and indexes.
.firebaserc	Selects the default Firebase project for CLI commands; it is a project identifier, not a service-account credential.
dalo/firebase.json	FlutterFire platform/app mapping for the Flutter Android project.
dalo/lib/firebase_options.dart	Generated Firebase client configuration used by Flutter. Keep project-specific local configuration out of public source control if ignored by the repository.
dalo/android/app/google-services.json	Generated Android Firebase client configuration; local setup file, ignored by Git.
dalo/android/app/src/main/AndroidManifest.xml	Android app metadata and camera, network, and vibration permissions.
dalo/android/*.gradle, settings.gradle, gradle/wrapper/	Android Gradle build setup and wrapper used to produce the APK.
AGENTS.md	Project-wide implementation rules, product requirements, visual design, and work-log instructions.
ANTIGRAVITY_PROMPT.md	Implementation-agent prompt covering the product flows, style, constraints, and delivery expectations.
README.md	Firebase setup, build instructions, roles, walkthroughs, security notes, and known limitations.
docs/CHANGELOG.md	Chronological record of app changes and their verification status.
docs/REVIEW_HANDOFF.md	Implementation handoff with project decisions, Firebase setup status, and remaining checks.
.gitignore	Excludes generated build/cache files and local Firebase configuration from normal commits.
functions/	Legacy Cloud Functions source retained for reference; the Spark app does not use it. Do not deploy it for this zero-budget setup.

Build and project constraints

- Build: from the dalo folder, run flutter pub get, then flutter build apk --debug. The output is dalo/build/app/outputs/flutter-apk/app-debug.apk.
- Before running the app from a fresh clone, configure Firebase Authentication (Email/Password), create Firestore, generate the FlutterFire Android configuration, and deploy the matching Firestore rules/indexes.
- The QR registration confirmation email is not implemented in the zero-budget Spark version. Firebase sends the separate account verification email; the event QR is shown in the app and can be shared.
- The project is Android-focused. iOS and web scanning are not implemented; scanning must use Daló's in-app camera.

A concise instructor demo

A short sequence that demonstrates the main product promise from event creation through attendance confirmation.

Order	Show	Narration cue
1	Sign in with a verified institution account.	Every account starts with student access; a verified account can also host an event.
2	Create an event with a capacity of two or three.	The event stores the owner, date/time, location, capacity, and current registration count.
3	Use a second verified account to find and register for the event.	The transaction creates the ticket and updates the count together, while refusing an already-full event.
4	Open the ticket and show the QR.	The token is unique to this event registration and contains no student email/name.
5	Open the event owner's attendee tools and scan the ticket in Daló.	A valid first scan marks attendance. Show the duplicate response after scanning again.
6	Show the live count or manual name search.	Hosts can review the attendee list, count, capacity, and manual check-in option.

Verification snapshot and limits

- The current source was reported to pass flutter analyze and flutter build apk --debug. The app was opened by the student developer; the latest code session did not have a connected device to independently repeat the sign-in routing flow.
- A prior local Firestore emulator check for a second verified account's registration/capacity transaction is recorded in the project handoff. Confirm the latest rules are deployed to the live Firebase project before presenting live event-owner access.
- Registration email containing the ticket QR remains pending. Do not present it as an active feature.
- This PDF describes the current project structure and implementation. It is a project guide, not a certification that every flow has been independently tested on multiple devices.

Closing summary

Daló combines student email verification, campus event hosting, capacity-aware registration, event-specific QR tickets, and one-time in-app attendance check-in in a small Flutter/Firebase Android app.

Prepared 16 October 2026 | Daló college project